

Rapport de Cadrage de Projet : Architecture Zero Trust avec IAM/PIM

1. Titre du projet

- **Variation 1 :** Architecture Zero Trust : Réduction de la surface d’attaque par une gestion d’identités et de privilèges basée sur le modèle IAM/PIM.
- **Variation 2 :** Conception et implémentation d’un socle d’authentification Zero Trust pour améliorer la conformité et la sécurité des actifs informationnels.
- **Variation 3 :** Intégration d’une solution IAM/PIM Open Source comme pilier d’une stratégie de sécurité Zero Trust : De la théorie au Proof of Concept.

2. Alternatives architecturales (Cloud vs On-Premise)

Le choix de la plateforme de gestion des identités et des accès (IAM) est un pilier fondamental de toute stratégie Zero Trust. Deux approches principales s'opposent :

- **Solution Cloud (SaaS - Software as a Service) :** Des plateformes comme Microsoft Entra ID (Azure AD), Okta ou Duo offrent des services IAM complets, managés et hautement disponibles. L'enjeu principal réside dans l'externalisation de la gestion de l'identité, impliquant une dépendance à un fournisseur tiers. Bien que le déploiement soit accéléré et la charge opérationnelle réduite, cela pose des questions de souveraineté des données, de flexibilité (enfermement propriétaire ou *vendor lock-in*) et de coûts récurrents (modèle par abonnement). L'interopérabilité est excellente mais souvent contrainte à l'écosystème du fournisseur.
- **Solution On-Premise / Auto-hébergée :** Des solutions Open Source comme Authentik, Keycloak ou FreeIPA permettent de construire une infrastructure IAM/PIM sur ses propres serveurs (physiques ou virtualisés) ou sur une infrastructure IaaS (Infrastructure as a Service) privée. Cette approche garantit une **souveraineté totale des données** et une flexibilité maximale pour l'intégration et la personnalisation. Cependant, elle induit une complexité de déploiement et de maintenance significativement plus élevée, nécessitant des compétences internes en administration système, en sécurité et en gestion de bases de données. La haute disponibilité et la scalabilité deviennent la responsabilité de l'équipe projet.

3. Étude comparative des solutions

Le tableau suivant synthétise les différences majeures entre les deux modèles.

Critère	Solution Commerciale (SaaS - ex: Entra ID)	Solution Open Source (Auto-hébergée - ex: Authentik)
Coût (TCO)	Coûts d'abonnement récurrents (OPEX), prévisibles mais potentiellement élevés à grande échelle.	Coût initial faible (licences gratuites), mais coûts cachés importants (CAPEX/OPEX) : matériel, maintenance, expertise humaine.

Critère	Solution Commerciale (SaaS - ex: Entra ID)	Solution Open Source (Auto-hébergée - ex: Authentik)
Flexibilité	Modérée. Les fonctionnalités sont définies par le fournisseur. Personnalisation limitée aux options de l'API et de la console.	Très élevée. Le code source est accessible. Intégrations sur-mesure et adaptations profondes sont possibles.
Complexité déploiement	Faible. Configuration via des interfaces graphiques assistées. Pas de gestion d'infrastructure sous-jacente.	Élevée. Nécessite le provisionnement de serveurs, la configuration de bases de données, réseaux, et la sécurisation de l'ensemble.
Souveraineté des données	Limitée. Les données d'identité et les métadonnées de connexion sont stockées et traitées par un tiers.	Totale. L'organisation contrôle entièrement le stockage physique et logique des données sensibles.
Support	Support professionnel garanti par le fournisseur, avec des SLA (Service Level Agreements) définis.	Support communautaire. Le support professionnel est souvent une option payante via des intégrateurs tiers.

4. La solution retenue : Authentik (IAM) + Tailscale ACL + moteur JIT Python

Suite aux contraintes du plan gratuit (SSO non couvert), la brique HashiCorp Boundary a été retirée du périmètre. La solution implémentée repose désormais sur :

- **Authentik** pour l'authentification et la gouvernance des identités (IAM).
- **Tailscale ACL** pour l'application réseau des droits d'accès.
- **Un moteur JIT développé en Python** pour l'orchestration des demandes, l'approbation, la révocation et l'expiration des accès.

1. Rôle de chaque composant

- **Authentik (IAM) :**
 - Vérifie l'identité utilisateur (SSO, groupes, permissions).
 - Fournit les claims nécessaires à l'application web (groupes, email, permissions).
- **Application Flask + DB PostgreSQL :**
 - Gère le cycle de vie des demandes d'accès (**pending**, **approved**, **denied**, **expired**).
 - Applique les règles métier :
 - blocage d'une nouvelle demande si une demande **pending** existe déjà,
 - accès autorisé si tag commun user/machine ou ACL approuvée active,
 - affichage du temps restant avant expiration des droits.
- **Moteur ACL Python (**ACL_work**) :**
 - **approve_access_request(...)** : applique la règle ACL et met à jour la DB.
 - **revoke_access_request(...)** : retire la règle ACL et met à jour la DB.

- `cleanup_expired_requests(...)` : supprime les droits expirés et passe les statuts en `expired`.

- **Cron de maintenance :**

- Exécute périodiquement le script d'expiration pour garantir le caractère temporaire des accès JIT.

2. Flux Zero Trust retenu

1. L'utilisateur s'authentifie via Authentik.
2. Il demande un accès à une machine depuis l'UI.
3. L'application vérifie en DB l'absence de demande `pending` active.
4. Un administrateur approuve la demande.
5. Le moteur Python applique la règle ACL Tailscale pour la machine cible.
6. La demande devient `approved` avec une date d'expiration.
7. Le cron retire automatiquement les droits à expiration et bascule le statut en `expired`.

5. Étapes de développement (Roadmap)

Le plan de réalisation du PoC se décompose comme suit :

1. Phase 1 - Audit et Conception :

- Définir le périmètre du laboratoire : 1-2 applications web (ex: Grafana) et 1-2 cibles d'infrastructure (ex: un serveur SSH, une base de données PostgreSQL).
- Cartographier les flux d'accès et rédiger les politiques de sécurité cibles (ex: "Seuls les administrateurs peuvent accéder au serveur SSH de production, après validation MFA et pour une durée de 1h maximum").

2. Phase 2 - Déploiement du socle IAM (Authentik) :

- Mise en place de l'environnement de virtualisation (Docker).
- Déploiement et configuration de base d'Authentik (utilisateurs, groupes).
- Intégration d'une application web avec Authentik (via Outpost) pour valider le SSO.

3. Phase 3 - Déploiement Tailscale + ACL :

- Configuration du tailnet et des tags machines.
- Validation des API Tailscale (`TAILSCALE_API_TOKEN`, `TAILSCALE_TAILNET`).
- Vérification des règles ACL de base et de leur lisibilité.

4. Phase 4 - Implémentation du moteur JIT Python :

- Développement des fonctions d'approbation/révocation/cleanup dans `ACL_work`.
- Intégration dans les routes Flask d'administration.
- Synchronisation statuts DB <-> ACL réseau.

5. Phase 5 - Intégration UI et gouvernance des demandes :

- Blocage du bouton de demande si une requête `pending` existe.

- Affichage des accès **approved** avec timer côté utilisateur.
- Contrôle métier : accès autorisé par tag commun ou ACL approuvée active.

6. Phase 6 - Exploitation et audit :

- Mise en place du cron d'expiration.
- Vérification de la traçabilité complète (qui a demandé, qui a approuvé, quand l'accès a expiré).

6. Fonctionnalités clés à implémenter (Use Cases)

Pour valider la pertinence de la solution, les cas d'usage suivants devront être développés :

- **UC-01 : SSO et MFA pour Application Web** : Un utilisateur accède à un dashboard Grafana. Il est redirigé vers Authentik, doit s'authentifier et valider un second facteur (MFA). Authentik confirme son identité et l'autorise à accéder à Grafana.
- **UC-02 : Accès Conditionnel basé sur le Contexte (IAM)** : Le même accès à Grafana est tenté depuis une adresse IP non approuvée. Authentik bloque la tentative d'authentification en amont, même si le mot de passe est correct.
- **UC-03 : Demande d'accès JIT à une machine Tailscale** : Un utilisateur non autorisé sur une machine crée une demande d'accès. L'application enregistre la demande en **pending** et empêche les doublons.
- **UC-04 : Approbation admin et activation ACL temporaire** : Un administrateur approuve la demande, le moteur Python applique la règle ACL Tailscale, la demande passe en **approved** avec expiration.
- **UC-05 : Expiration automatique et audit** : Le cron supprime les droits à échéance, marque la demande en **expired**, et les journaux permettent de tracer tout le cycle de vie de l'accès.